

PERTEMUAN 5

IMPLEMENTASI PERANCANGAN PERANGKAT LUNAK

MATERI PERTEMUAN 5

1. Class Diagram
2. Deployment Diagram
3. Implementasi dalam Bahasa Pemrograman

I. Class Diagram

a. Pengertian *Class Diagram*

Menurut Tohari(2013:83), **diagram kelas (*class diagram*)** menggambarkan jenis-jenis dari objek dalam suatu sistem dan berbagai jenis hubungan statis yang ada diantaranya.

Menurut (Akil, 2018), ***Class (Kelas)*** adalah deskripsi dari satu set objek-objek yang berbagi atribut-atribut, operasi-operasi, relasi-relasi, semantik-semantik yang sama.

Class Diagram (Lanjutan)

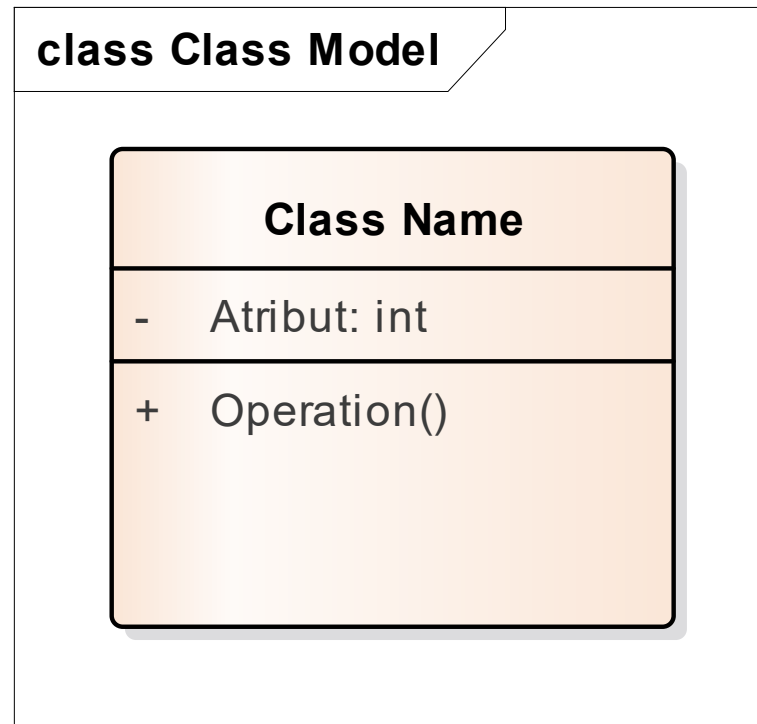
Sebuah Kelas merupakan kumpulan dari objek yang memiliki karakteristik yang sama seperti atribut, operasi hubungan dan semantik.

Kelas juga merupakan sebuah spesifikasi yang jika diinstansiasi akan menghasilkan sebuah objek dan merupakan inti dari pengembangan dan perancangan berorientasi objek.

b. Simbol-Symbol dalam *Class* Diagram

1. *Class*

Class terdiri dari tiga area, sebagai berikut :



b. Simbol-Simbol dalam *Class Diagram* (Lanjutan)

Keterangan

- ***Class Name***

- ☐ Nama kelas bersifat tekstual dan selalu diawali dengan huruf besar
- ☐ Jika terdiri dari dua suku kata, maka penamaannya harus disambung dan suku kata kedua diawali dengan huruf besar
- ☐ Kelas bisa digunakan untuk mewakili orang, tempat atau hal-hal yang dibutuhkan oleh sistem, yang digunakan untuk menangkap atau menyimpan informasi

b. Simbol-Symbol dalam *Class Diagram* (Lanjutan)

Keterangan

- ***Attribute* (Atribut)**

- ☐ Atribut adalah properti yang memiliki nama dari suatu kelas yang menjadi karakteristik dari kelas tersebut
- ☐ Atribut merepresentasikan properti yang sedang anda modelkan
- ☐ Sebuah atribut harus memiliki type data sesuai dengan bahasa pemrograman kelas tersebut
- ☐ Nama atribut biasanya diawali dengan huruf kecil dan suku kata berikutnya diawali huruf besar

b. Simbol-Symbol dalam *Class Diagram* (Lanjutan)

Keterangan

- ☐ Atribut juga memiliki jangkauan visibility, baik yang bersifat public (+), private (-), protected (#) dan package (~)
- ☐ Visibility adalah spesifikasi yang dapat dimiliki oleh kelas, atribut dan operasi yang menentukan hak akses atau jangkauan atribut dan operasi tersebut
- ☐ (-) private, hanya kelas yang bersangkutan yang dapat menggunakan fitur tersebut

b. Simbol-Simbol dalam *Class Diagram* (Lanjutan)

- ❑ (+) public, kelas-kelas diluar kelas yang bersangkutan dapat mengakses dan menggunakannya
- ❑ (#) protected, setiap kelas turunannya dapat mengakses dan menggunakannya
- ❑ (~) package, hanya kelas yang berada dalam package yang sama yang dapat mengakses dan menggunakannya

b. Simbol-Simbol dalam *Class Diagram* (Lanjutan)

Keterangan

- ***Operation()***

- ☐ *Operation* (operasi) adalah implementasi dari layanan yang dapat diminta dari objek manapun dari kelas
- ☐ Operasi merupakan perilaku dari objek yang mempengaruhi status dari objek
- ☐ Dalam konteks pemrograman operasi pada dasarnya sebuah fungsi atau prosedur yang dimiliki objek terkait
- ☐ Suatu operasi dapat mengembalikan nilai sesuai dengan type data yang kita tentukan

Contoh *Class*

class Class Model

Mobil

- merk: String
 - mesin: String
 - model: String
 - tahun: int
 - warna: String
-
- + break(): void
 - + moveBackward(): void
 - + moveForward(): void
 - + moveLeft(): void
 - + moveRight(): void

```
public class Mobil {
```

```
    private String merk;  
    private String mesin;  
    private String model;  
    private int tahun;  
    private String warna;
```

```
    public Mobil() {
```

```
    }
```

```
    public void finalize()  
        throws Throwable {
```

```
    }
```

```
public void break() {
```

```
    }
```

```
    public void moveBackward() {
```

```
    }
```

```
    public void moveForward() {
```

```
    }
```

```
    public void moveLeft() {
```

```
    }
```

```
    public void moveRight() {
```

```
    }
```

```
}
```

2. *Multiplicity*

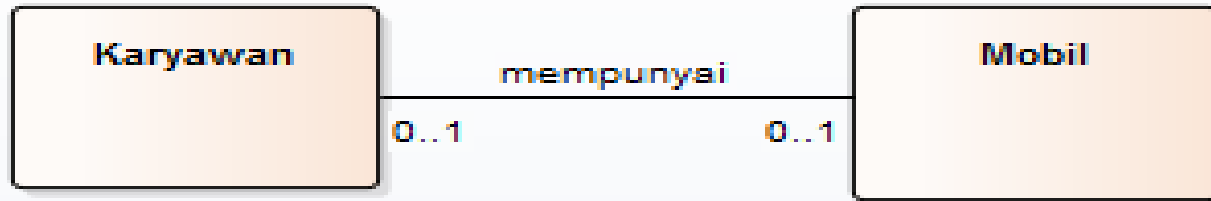
Menurut (Akil, 2018), *Multiplicity* (baca : multiplisitas) adalah spesifikasi jangkauan kardinalitas yang diijinkan yang dimiliki suatu entitas.

Multiplisitas menunjukkan jumlah suatu objek yang dapat berhubungan dengan objek yang lain.

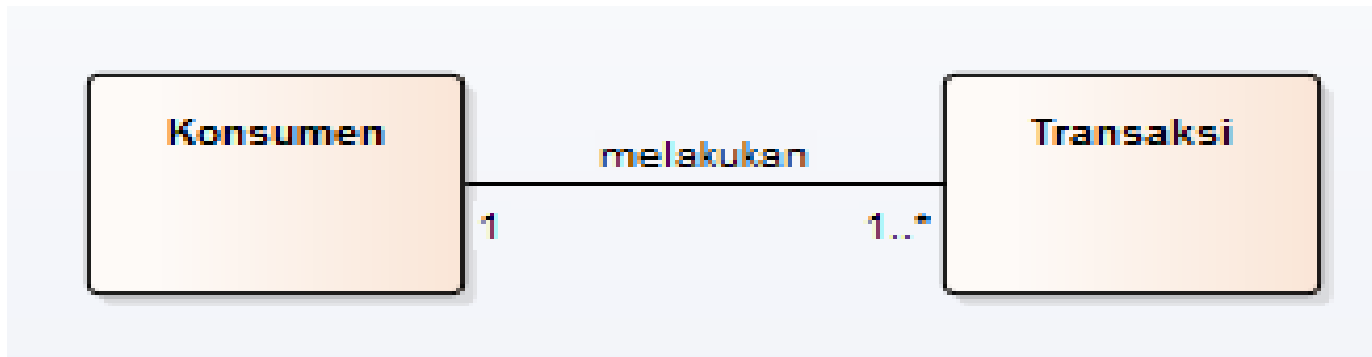
Multiplisitas biasanya ditunjukkan dengan satu atau banyak, tetapi secara khusus dapat ditunjukkan pula dengan bilangan integer lebih besar atau sama dengan nol

Indikator	Arti
0..1	Nol atau satu
1	Hanya satu
0..*	Nol atau banyak
1..*	Satu atau banyak
n	Hanya n (dengan $n > 1$)
0,..n	Nol sampai n (dengan $n > 1$)
1..n	Satu sampai n (dengan $n > 1$)

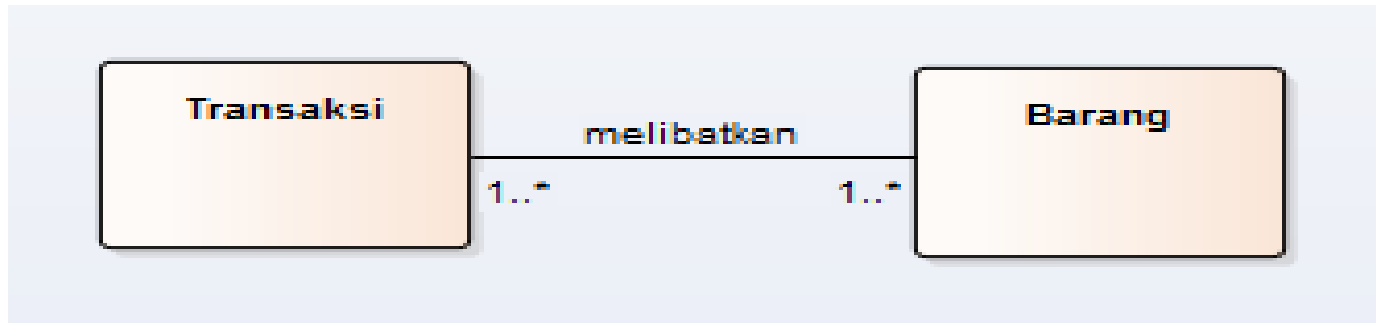
Contoh



- (Cara baca dari kiri ke kanan) Setiap karyawan bisa kemungkinan tidak memiliki mobil atau memiliki satu mobil
- (Cara baca dari kanan ke kiri) Setiap mobil kemungkinan tidak dimiliki oleh karyawan atau dimiliki oleh seorang karyawan
- Kesimpulan kardinalitasnya adalah *one to one*



- (Cara baca dari kiri ke kanan) Setiap konsumen melakukan satu atau banyak transaksi
- (cara baca dari kanan ke kiri) Setiap transaksi dilakukan oleh satu konsumen
- Kesimpulan kardinalitasnya adalah *one to many*



- (Cara baca dari kiri ke kanan) Setiap transaksi melibatkan satu atau banyak barang
- (Cara baca dari kanan ke kiri) Setiap barang dilibatkan dalam satu atau banyak transaksi
- Kesimpulan kardinalitasnya adalah *many to many*

3. **Generalization**

Menurut (Akil, 2018)

- Sebuah **generalization** (generalisasi) adalah hubungan antara kelas umum (*parent / superclass*) dengan kelas khusus (*child/ subclass*)
- Dengan hubungan generalisasi suatu kelas *child* dapat mewarisi struktur dan tingkah laku dari kelas *parentnya*.

4. *Association (Asosiasi)*

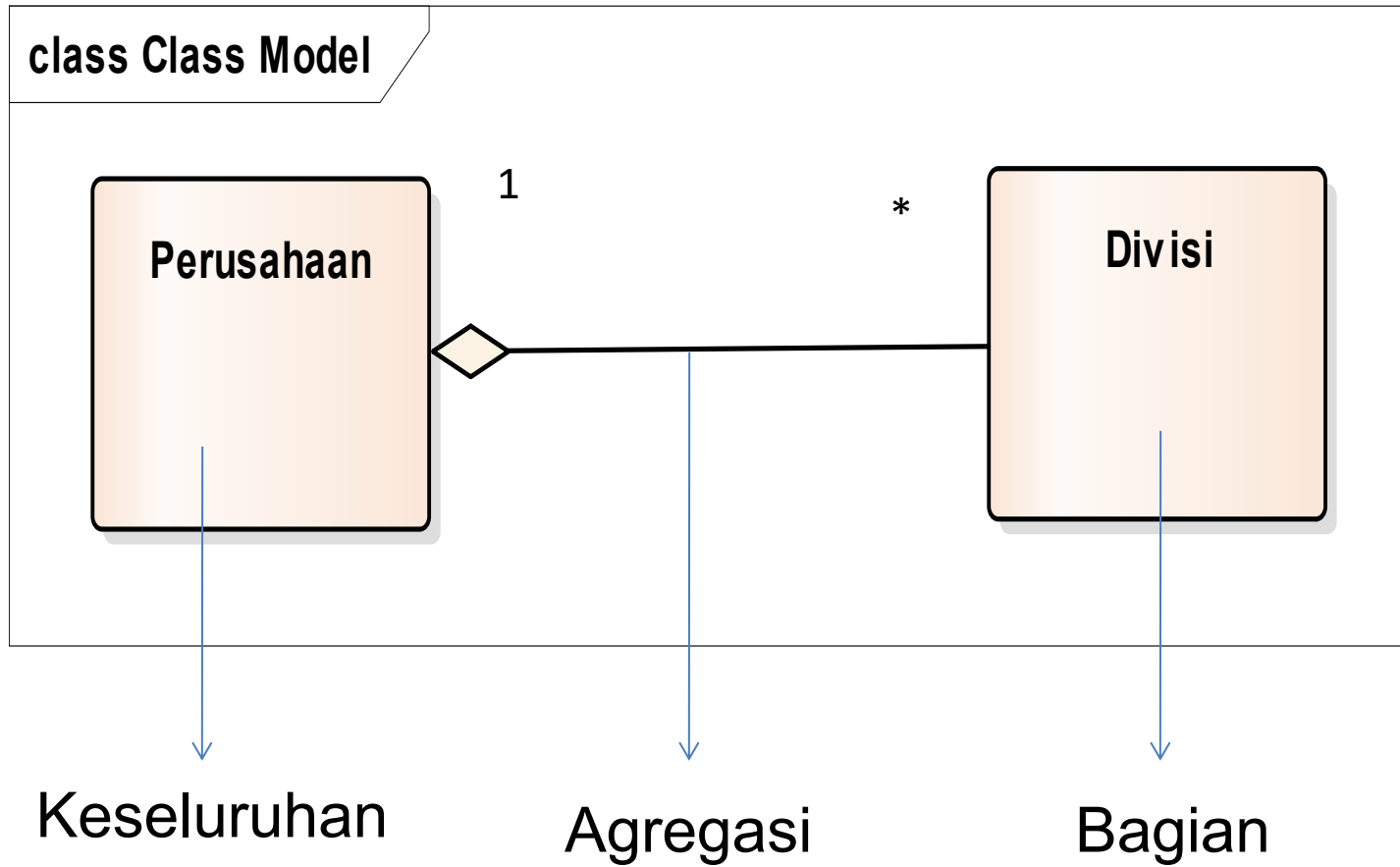
Menurut (Akil, 2018)

- **Asosiasi** adalah hubungan struktural yang menspesifikasikan bahwa objek-objek dari satu hal terhubung dengan objek yang lainnya.
- Sebuah asosiasi biasanya memiliki nama yang menjelaskan secara alami hubungan tersebut
- Dapat ditambahkan direction (arah) hubungan asosiasi tersebut

5. *Aggregation* (Agregasi)

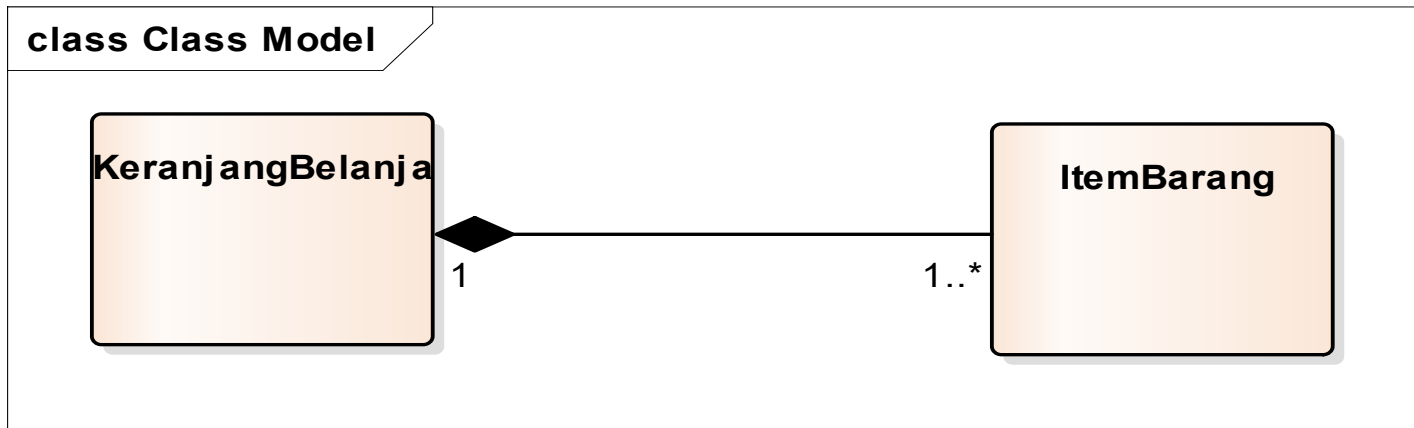
- Hubungan keseluruhan/bagian, dimana satu kelas merepresentasikan hal yang lebih besar keseluruhan yang berisi hal lebih kecil bagian, hubungan ini disebut Agregasi
- Agregasi merepresentasikan hubungan memiliki yang berarti bahwa objek keseluruhan memiliki objek-objek bagian.
- Agregasi digambarkan sebagai asosiasi dengan belah ketupat kosong pada objek keseluruhan

Contoh



6. *Composition* (komposisi)

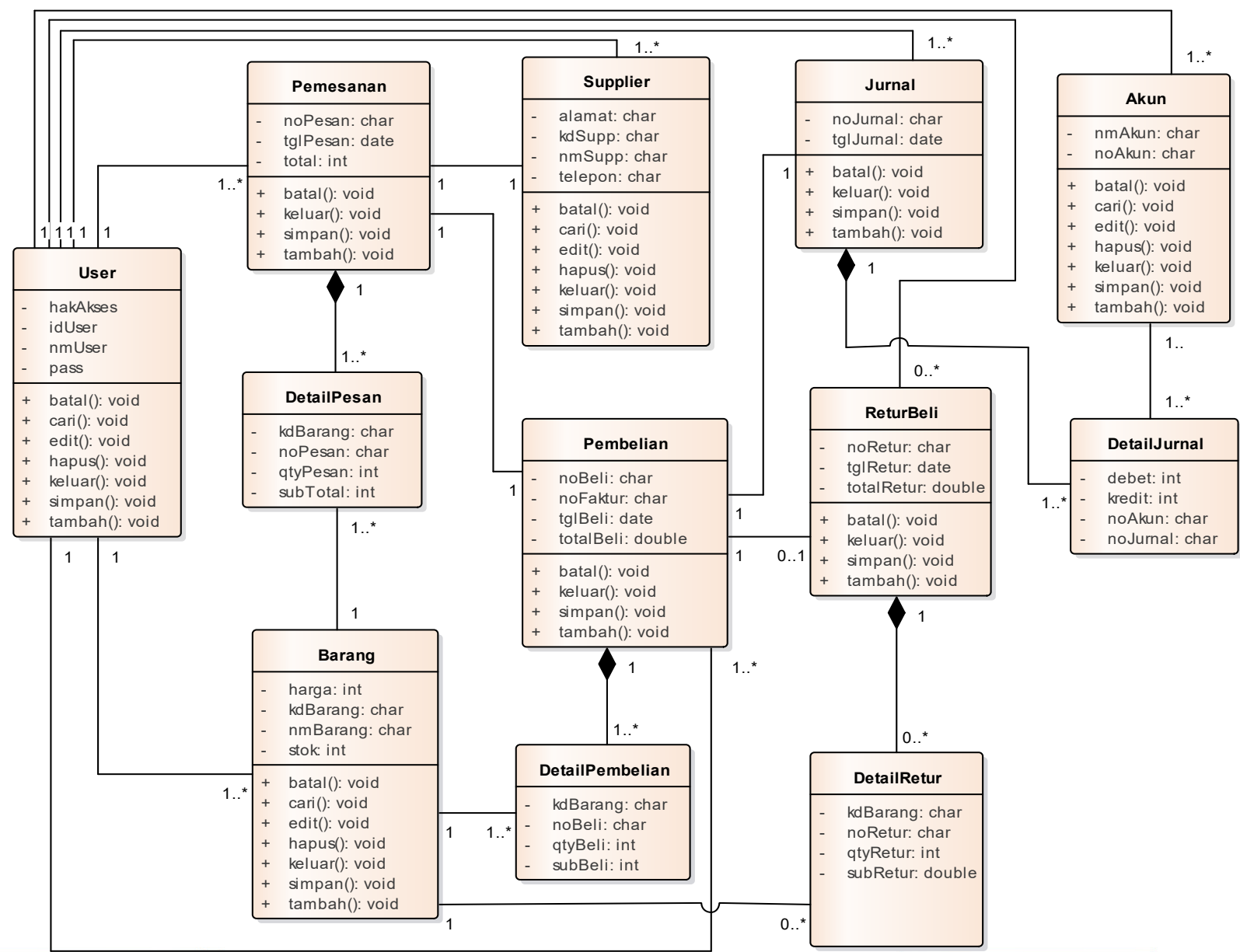
- Komposisi adalah bentuk agregasi dengan hubungan kepemilikan yang lebih kuat dan lifetimedari objek sebagai bagian dari keseluruhan.
- Objek-objek yang merupakan bagian pada saat diinstansiasi hanya dapat dimiliki oleh objek keseluruhan dan hidup selama keseluruhan hidup.



- Dari gambar diatas, objek KeranjangBelanja merupakan komposisi dari objek ItemBarang
- Objek ItemBarang sepenuhnya dimiliki oleh objek keranjangBelanja dan hidup selama objek KeranjangBelanja hidup

Class Diagram yang terbentuk untuk kasus sistem pembelian sebagai berikut :

class Sistem Pembelian



II. Deployment Diagram

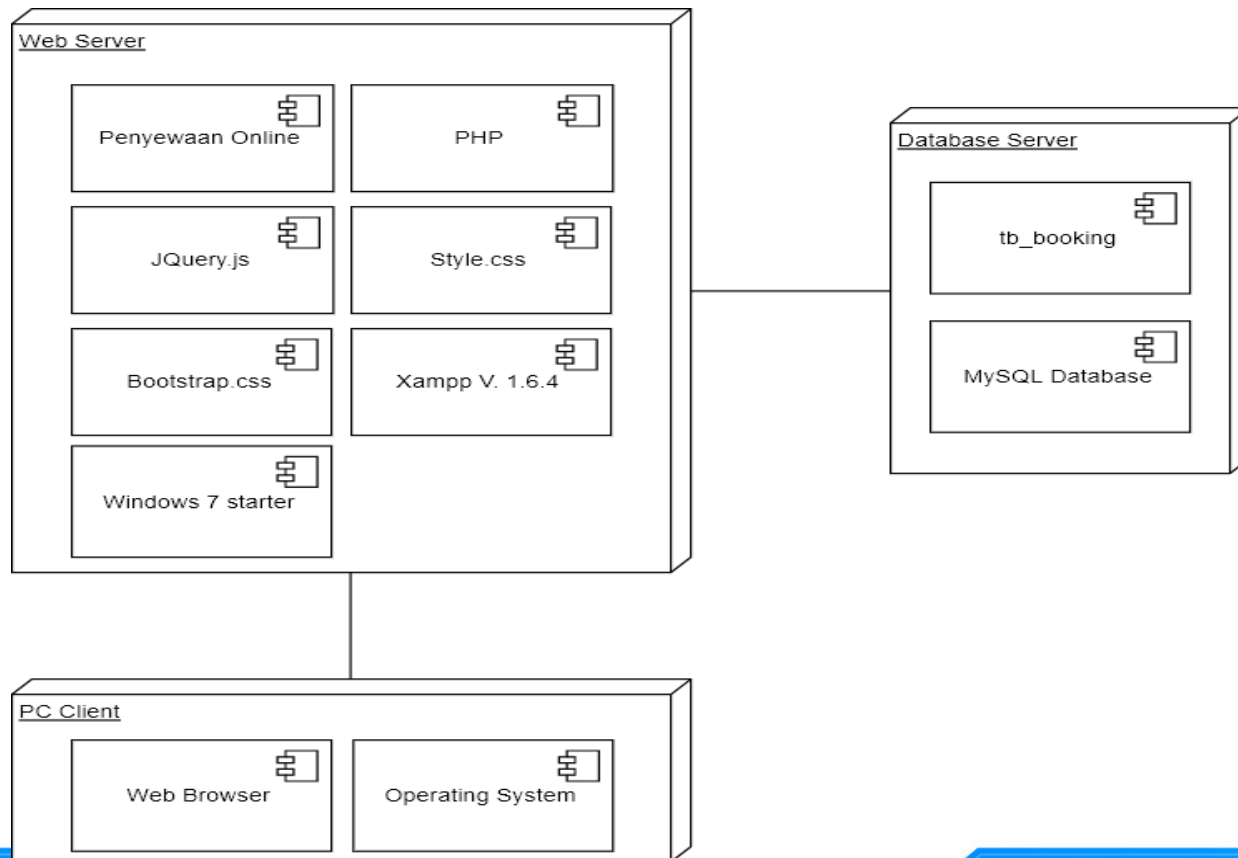
a. Pengertian Deployment Diagram

- Menggambarkan topologi hardware yang ada pada sistem dan merupakan bagian dari spesifikasi arsitektur sistem.
- Menggambarkan penampakan statis dari detail implementasi komponen software yang terkait dengan hardware.
- Menggambarkan detail konfigurasi run-time yang didalamnya termasuk detail hardware seperti processor dan perangkat lain, protokol, middleware dan server database yang didistribusikan di jaringan atau server database terpusat.

Deployment Diagram

Deployment diagram menunjukkan susunan fisik sebuah sistem, menunjukkan bagian perangkat lunak mana yang berjalan pada perangkat keras tersebut.

Contoh:



b. Simbol Deployment Diagram

a. Node:

- Merupakan unit komputasi yang dapat berupa hardware maupun software.
- Contoh: komputer, printer, mesin ATM, Card Reader.
- Disimbolkan dengan kubus

Node:

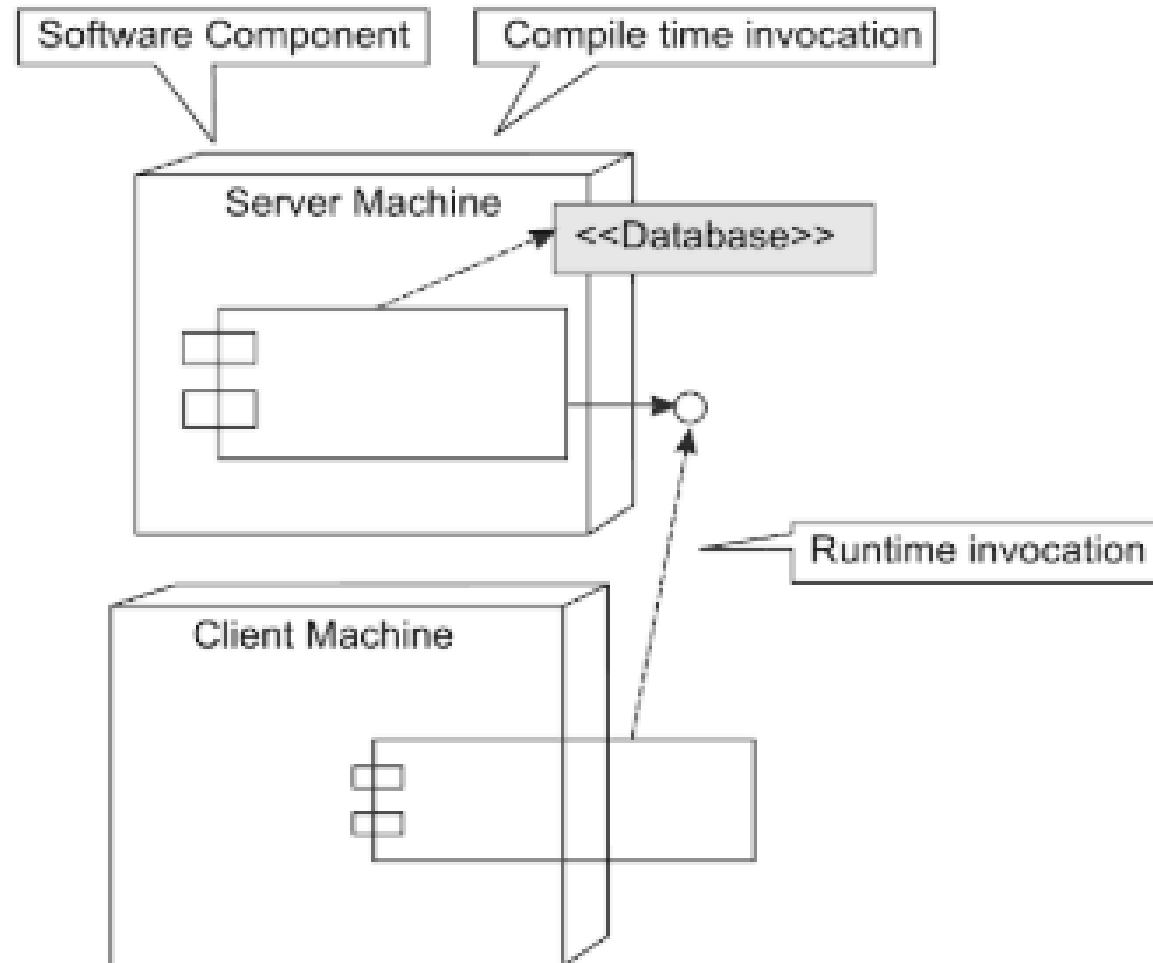
- Kubus tersebut menggambarkan device ataupun processor
- Device digambarkan dengan kubus tanpa bayangan.
- Processor digambarkan dengan kubus berbayangan



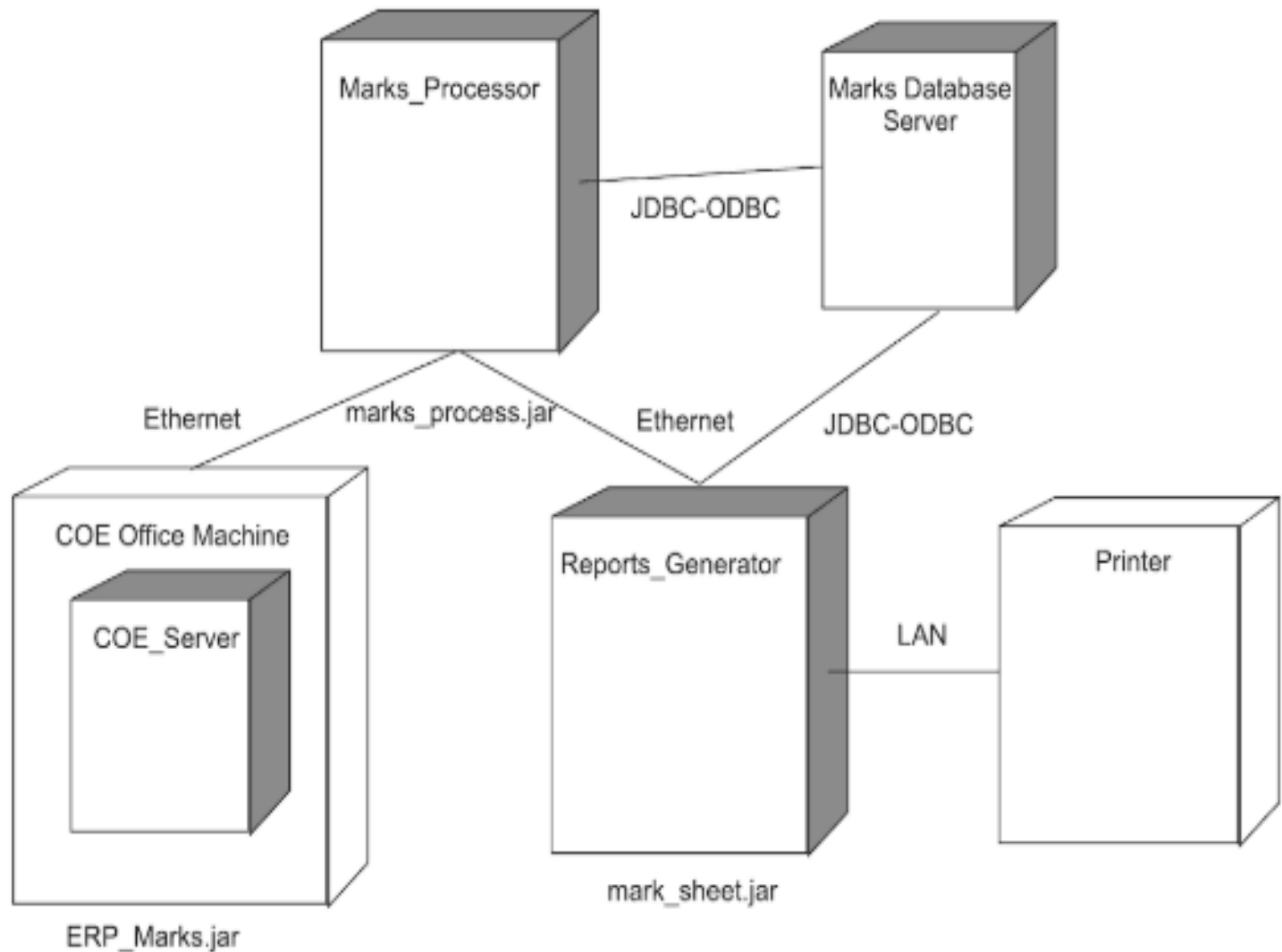
b. Connection:

- Menggambarkan interaksi antar device.
- Secara umum hubungan antara nodes menggambarkan mekanisme komunikasi dan dapat bertindak sebagai protokol software
- Menggambarkan kabel RS-232, koneksi Ethernet, koneksi LAN, satelit, RMI, WAP, SOAP, ODBC, JDBC-ODBC.
- Disimbolkan dengan garis lurus horizontal

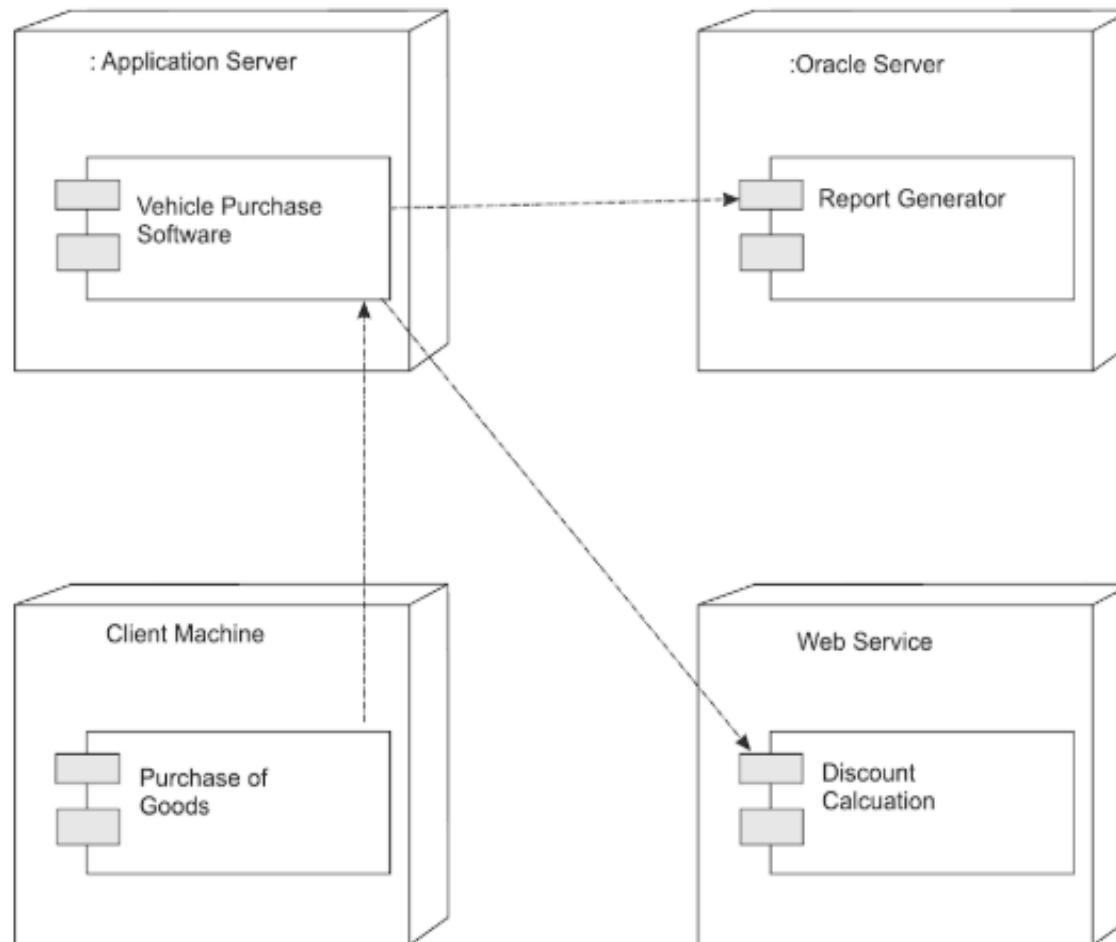
Contoh Deployment diagram dengan komponen software dan database.



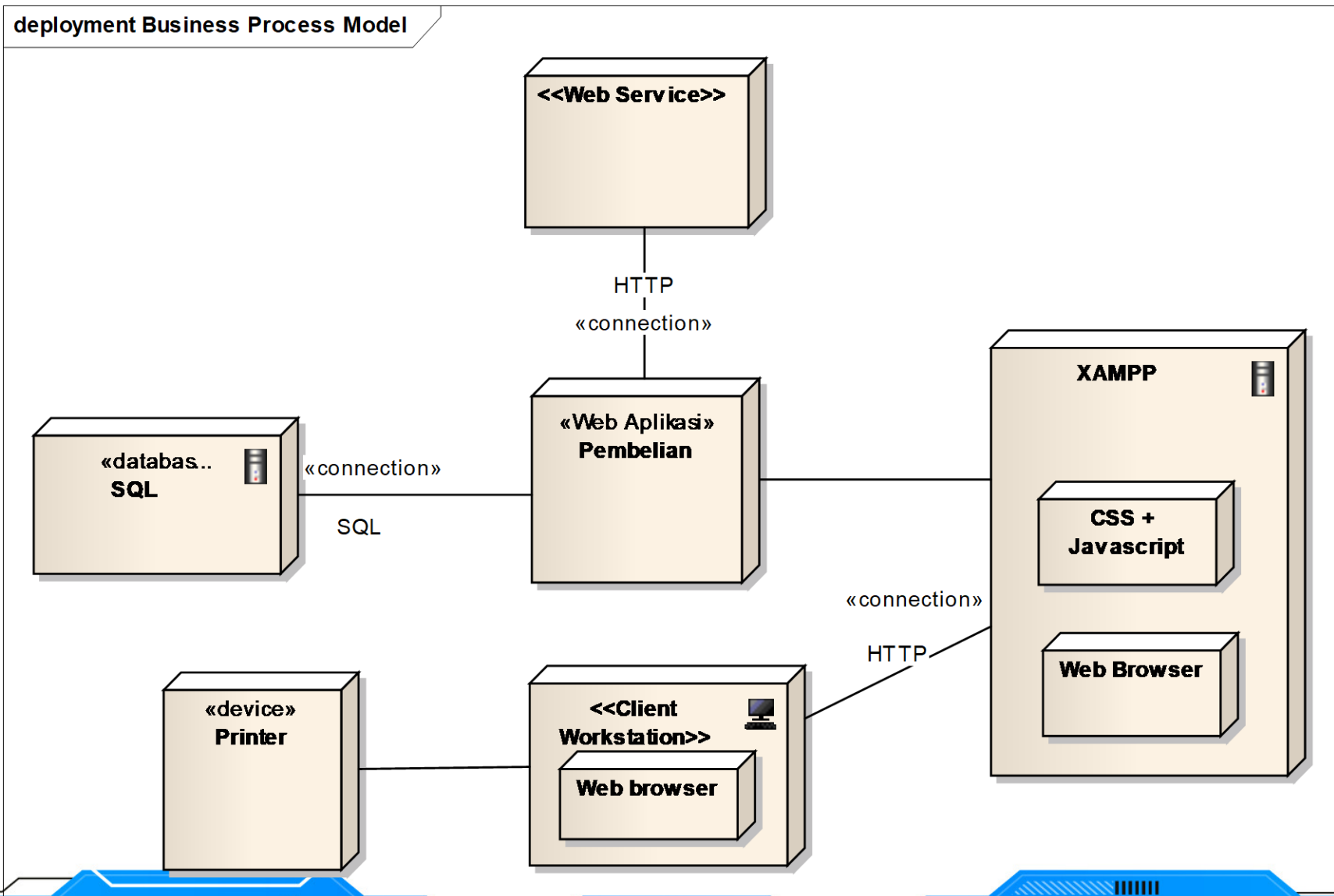
Contoh pada ERP System



Contoh koneksi antara PC user, server aplikasi, server database dan web service



Deployment Diagram untuk Sistem Pembelian



III. BAHASA PEMROGRAMAN

- Contoh bahasa PBO:
Java, C++, C#, Python, PHP, Ruby, Perl, Object Pascal, ObjectiveC, Dart, Swift, Scala, Common Lisp dan Smalltalk.
- Meningkatkan efektivitas dan efisiensi program karena adanya konsep penggunaan ulang komponen (*Component Reuseable*)”

Implementasi Class Diagram

- Bisa dalam bentuk kelas di dalam bahasa pemrograman ataupun kelas-kelas persisten yang hadir dalam tabel di dalam basisdata relasional.

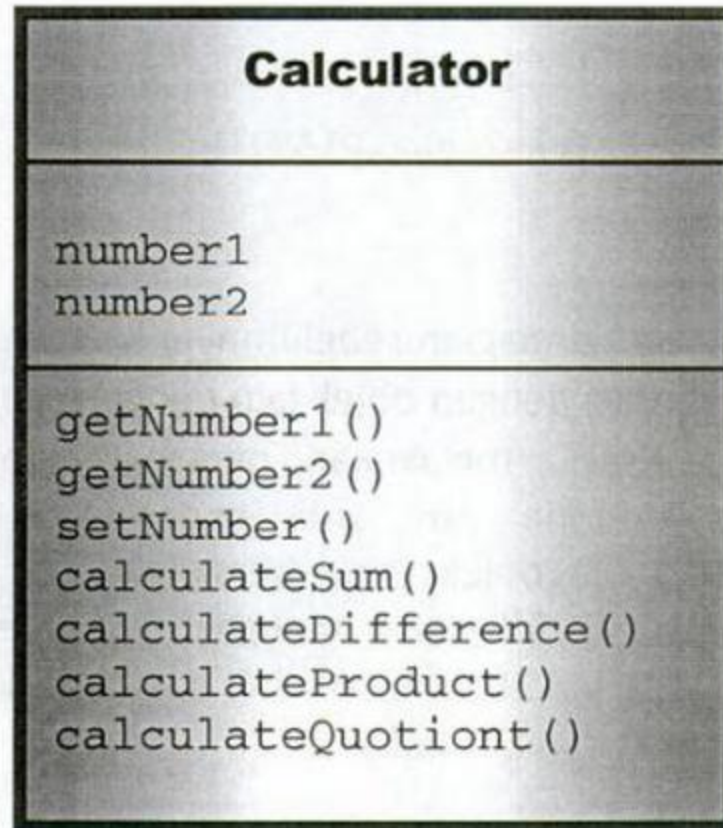
Implementasi Class Diagram (Lanjutan)

- Membuat implementasi kode atas kelas-kelas yang terdapat pada class diagram.
- Membuat konstruktor ataupun *setter* dan *getter* untuk kelas yang bersangkutan.
- Implementasi atribut dalam kelas menjadi kode program, dan sesuai dengan konsep PBO yakni *encapsulation* maka setiap atribut memiliki visibility private.

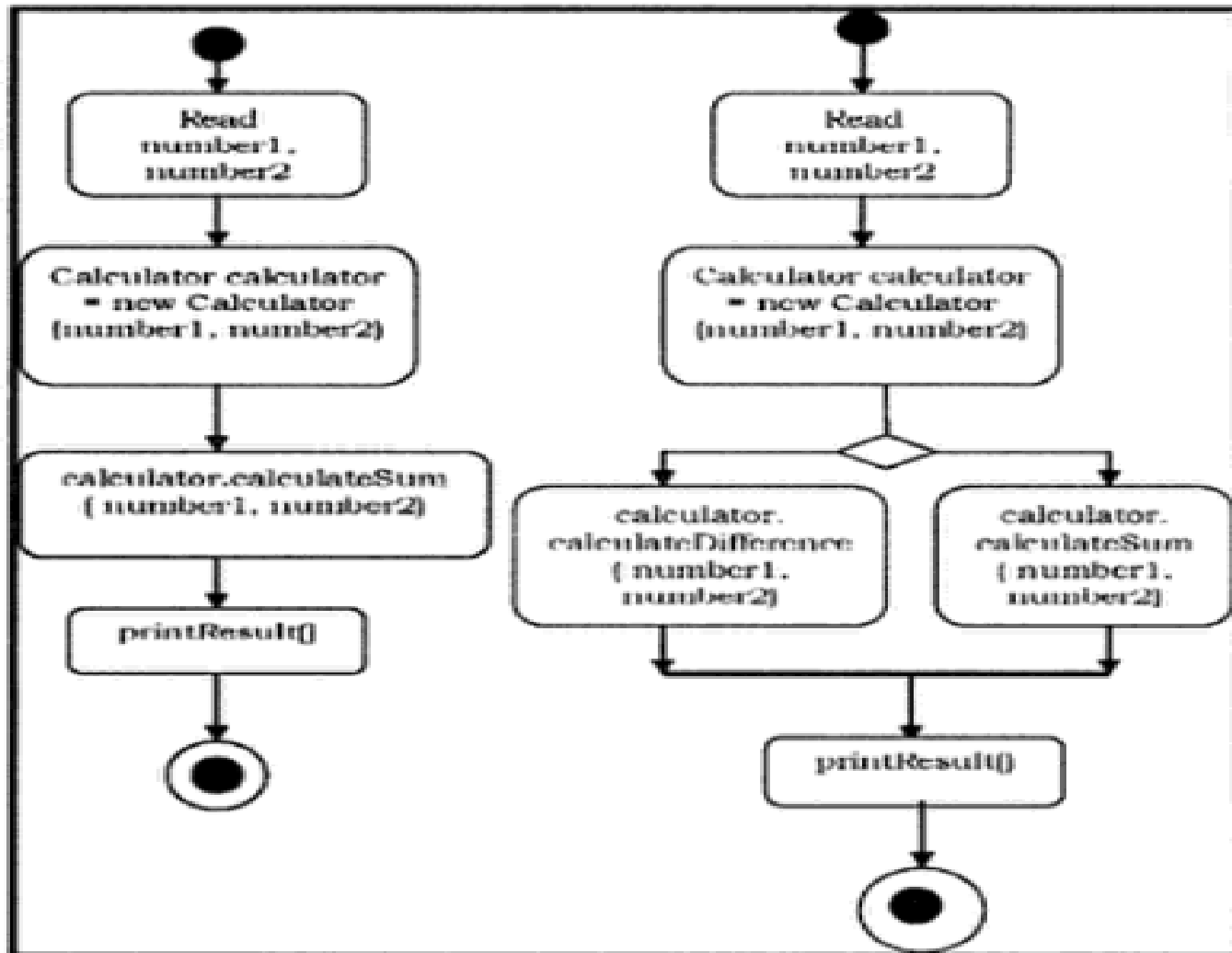
Implementasi Class Diagram (Lanjutan)

- Membuat implementasi kode atas setiap metode pada setiap kelas dengan memberikan visibility public
- Mengimplementasi atribut dan metode yang akan diwariskan dengan memberikan visibility protected.

Implementasi Class Diagram (Lanjutan)



Implementasi Activity Diagram




```
class Calculator {
private double number1;
private double number2;
public Calculator(double initNumber1 , double initNumber2) {
number1 = initNumber1;
number2 = initNumber2;
}

public double getNumber1 () {
return number1;
}
public double getNumber2() {
return number2;
}
public void setNumbers(double newNumber1 , double
newNumber2) {
number1 = newNumber1;
number2 = newNumber2;
}

public double calculateSum() {
return number1 + number2;
}
public double calculateDifference() {
return number1 - number2;
}
public double calculateProduct() {
return number1 * number2;
}

public double calculateQuotient() {
/* Pembagian dengan nol akan menghasilkan
* hasil-hasil sebagai berikut :
* Saat numerator dan denominator bernilai 0 :
* Hasil adalah Double.NaN ("not-a-number")
* Saat denominator bernilai 0 :
* Hasil : Double. NEGATIVEINFINITY
* atau Double.POSITIVEINFINITY.
* Mencetak nilai-nilai sebagai "NaN" atau "Infinity".
return number1 / number2;
}
```

TUGAS 3 (Individu)

Membuat:

1. Class Diagram
2. Deployment Diagram

berdasarkan kasus yang diberikan oleh Dosen.

Jawaban tugas dikumpulkan pada pertemuan 6